

GIỚI THIỆU

Lập trình đối tượng phân tán là một trong những vấn đề nóng bỏng của công nghệ phân tán phần mềm ngày nay. Java là ngôn ngữ đi tiên phong trong việc giải quyết vấn đề lập trình phân tán. Chương này sẽ giới thiệu với bạn khái niệm và cách cài đặt các đối tượng phân tán bằng kỹ thuật RMI trong Java. Đây là kỹ thuật xuyên suốt trong toàn bộ kiến trúc java sau này.

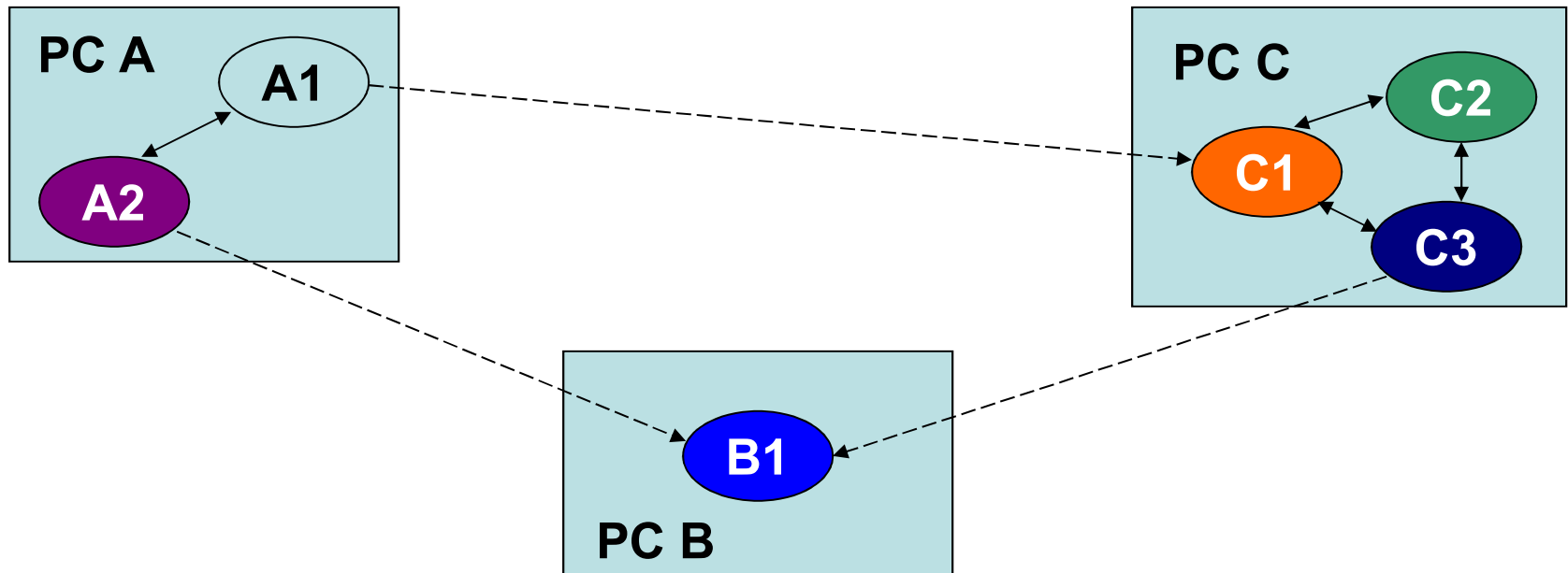
Các vấn đề chính sẽ được đề cập đến:

- Khái niệm về lập trình phân tán đối tượng và kỹ thuật RMI.
- Thiết kế ứng dụng phân tán RMI.
- Chuyển dữ liệu và tham chiếu đối tượng trên mạng.
- Kết nối mạng và vấn đề tường lửa (firewall).
- Web Services/SOAP một công nghệ và giao thức mới cho môi trường phân tán.

RMI VÀ LẬP TRÌNH PHÂN TÁN ĐỐI TƯỢNG

Thông thường các chương trình của chúng ta được viết dưới dạng thủ tục hoặc hàm và việc các hàm gọi lẫn nhau, truyền tham số hay kết quả cho nhau chỉ xảy ra ở máy cục bộ. Kỹ thuật RMI (Remote Method Invoke) – mang ý nghĩa triệu gọi phương thức từ xa – là cách thức giao tiếp giữa các đối tượng trong Java có mã lệnh cài đặt nằm trên các máy khác nhau có thể triệu gọi lẫn nhau.

Mô hình triệu gọi các đối tượng từ xa



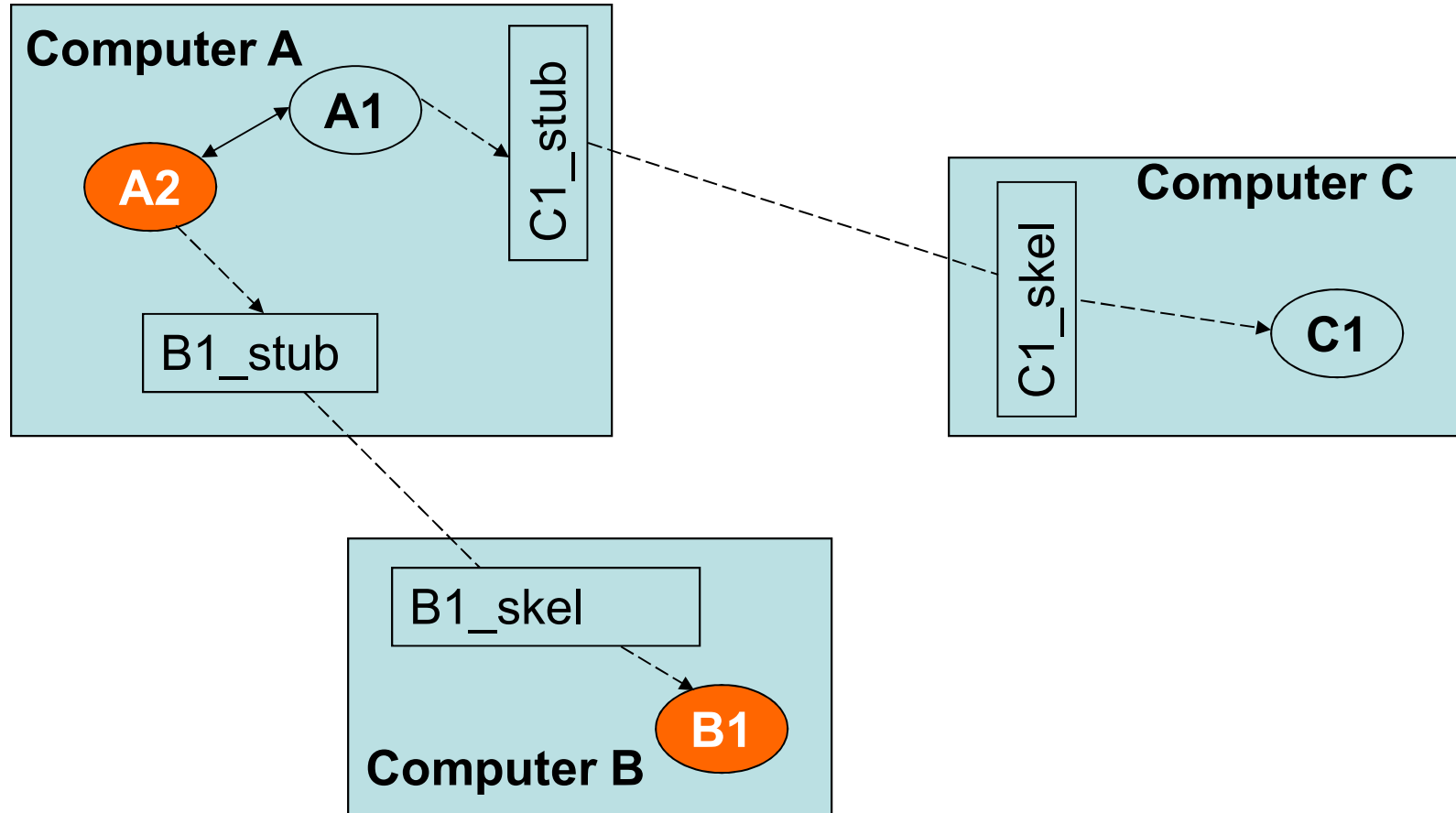
GỌI PHƯƠNG THỨC TỪ XA VÀ CÁC VẤN ĐỀ PHÁT SINH

- Việc gọi phương thức của đối tượng từ xa luôn phức tạp hơn gọi phương thức cục bộ.
- Các đối tượng trên hai máy khác nhau hoạt động trên hai tiến trình khác nhau nên việc tham chiếu đến biến địa chỉ hoàn toàn khác nhau.
- Lời gọi phương thức từ xa phải thông qua mạng và có thể bị ngắt ngang do mạng gặp sự cố.
- Các tham số truyền cho đối tượng ở xa phải được đóng gói và truyền qua mạng để đến với phương thức thực sự.

VAI TRÒ CỦA CÁC LỚP TRUNG GIAN (STUB VÀ SKELETON)

Để giải quyết vấn đề trên, đối tượng trên hai máy khác nhau không gọi trực tiếp mà thông qua lớp trung gian. Lớp này tồn tại ở cả hai phía Client và Server. Lớp ở máy Client gọi là **Stub**, lớp ở máy Server gọi là **Skel** (Skeleton).

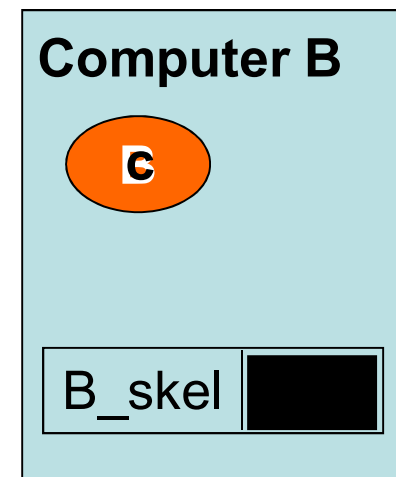
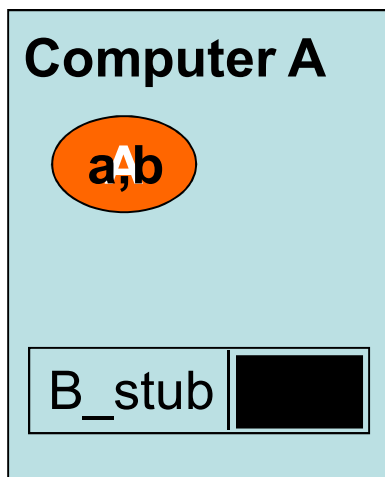
Ví dụ 1:



VAI TRÒ CỦA CÁC LỚP TRUNG GIAN (STUB VÀ SKELETON)

Ví dụ 2:

Phương thức A truyền cho phương thức B hai số a,b.
Phương thức B sẽ cộng hai số a,b cho ra kết quả c và trả về phương thức A. Quá trình diễn ra như sau:



$$a + b = c$$

CÀI ĐẶT ỨNG DỤNG PHÂN TÁN RMI

Các phương thức sử dụng trong chương trình:

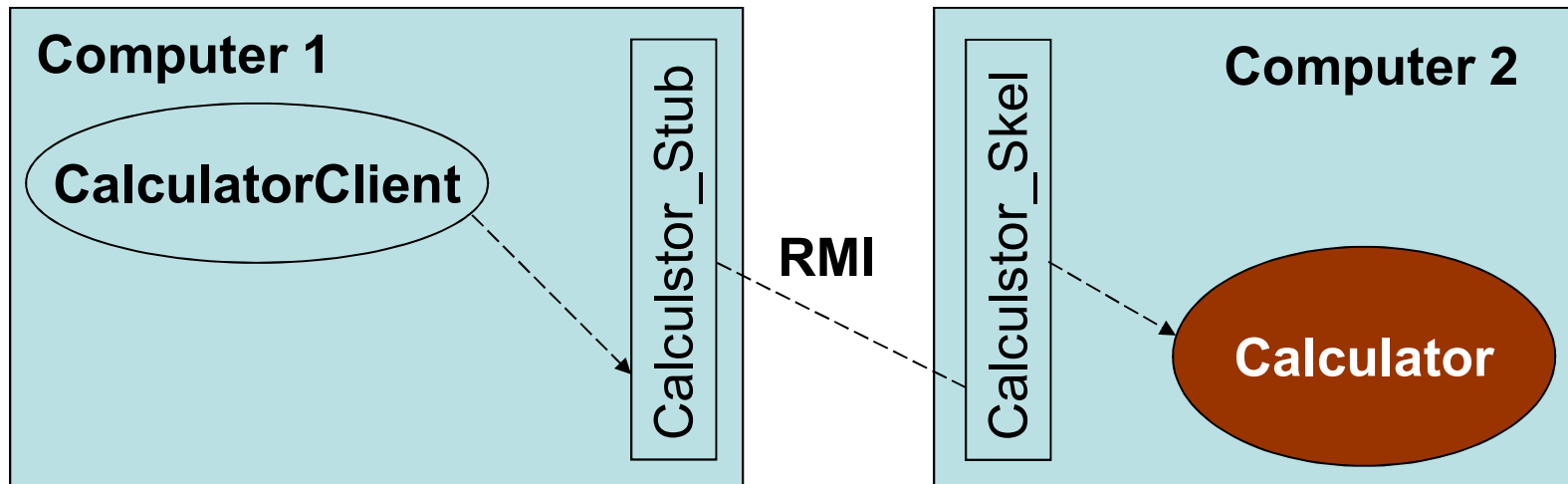
- ☑ **exportObject(Object)** : nằm trong lớp **UnicastRemoteObject**.
Phương thức này làm cho máy ảo Java nhận diện được đối tượng **Object**.
Để sử dụng được lớp **UnicastRemoteObject** phải khai báo: **import java.rmi.server.*;**
- ☑ **bind(URLString, Object)** thuộc lớp **Naming**.
 - **URLString**: Chuỗi định vị có dạng như sau:
rmi://hostName:port/ObjectName. Trong đó:
rmi: là tên giao thức dùng để đăng ký.
hostName, port: là địa chỉ IP và số hiệu cổng nơi máy chủ nơi bộ đăng ký đối tượng rmi đang chạy.
ObjectName: là tên bất kỳ gọi nhớ để đặt cho đối tượng. Các chương trình máy khách sẽ dựa vào tên này để truy tìm tham chiếu đến đối tượng cần dùng.
 - **Object**: Tên đối tượng.Phương thức **bind()** dùng để đăng ký một tên gọi nhớ cho đối tượng **Object** với bộ quản lý rmi.

CÀI ĐẶT ỨNG DỤNG PHÂN TÁN RMI

- ☑ **lookup(rmi://hostName:port/ObjectName).** Phương thức này thuộc lớp **Naming**. Đối số là chuỗi định dạng cho biết địa chỉ máy chủ và tên đăng ký đối tượng.

CÀI ĐẶT ỨNG DỤNG PHÂN TÁN RMI

☛ Cài đặt chương trình cộng hai số nguyên.



Hình mô tả triệu gọi đối tượng RMI giữa trình khách và đối tượng chủ ở xa.

CÀI ĐẶT ỨNG DỤNG PHÂN TÁN RMI

// Calculator.java

```
2 import java.rmi.*;
3
4 public interface Calculator extends Remote{
5     public int addNum(int x, int y) throws RemoteException;
6 }
```

Đánh dấu cho máy ảo Java biết khả năng giao tiếp với các đối tượng ở xa của Calculator

Từ lớp giao tiếp Calculator đối tượng thực sự được cài đặt như sau:

// Calculator.java

```
1 import java.rmi.*;
2
3 public class CalculatorImpl implements Calculator {
4
5     public int addNum(int x, int y) throws RemoteException{
6
7         System.out.println("Client request to calculate");
8
9         return (x+y);
10    }
11 }
```

CÀI ĐẶT ỨNG DỤNG PHÂN TÁN RMI

// CalculatorServer.java

```
1 import java.rmi.server.*;
2 import java.rmi.*;
3
4 public class CalculatorServer{
5     public static void main(String args[]) {
6         try{
7             CalculatorImpl c=new CalculatorImpl();
8
9             System.out.println("Exporting Calculator ...");
10
11             UnicastRemoteObject.exportObject(c);
12
13             Naming.bind("rmi://localhost/MyCalculator", c);
14
15             System.out.println("Register Calculator!");
16
17         }catch (Exception e){
18
19             System.out.println(e);
20
21         }
22     }
```

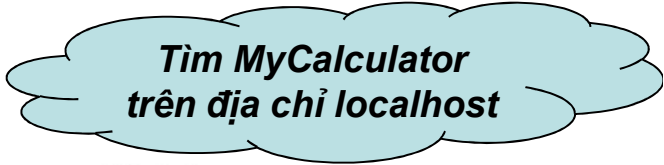
*Thông báo sự hiện diện
của c cho máy ảo Java*

*Đăng ký c với bộ
quản lý RMI. Tên gọi
nhớ của c là
MyCalculator.*

CÀI ĐẶT ỨNG DỤNG PHÂN TÁN RMI

//CalculatorClient.java

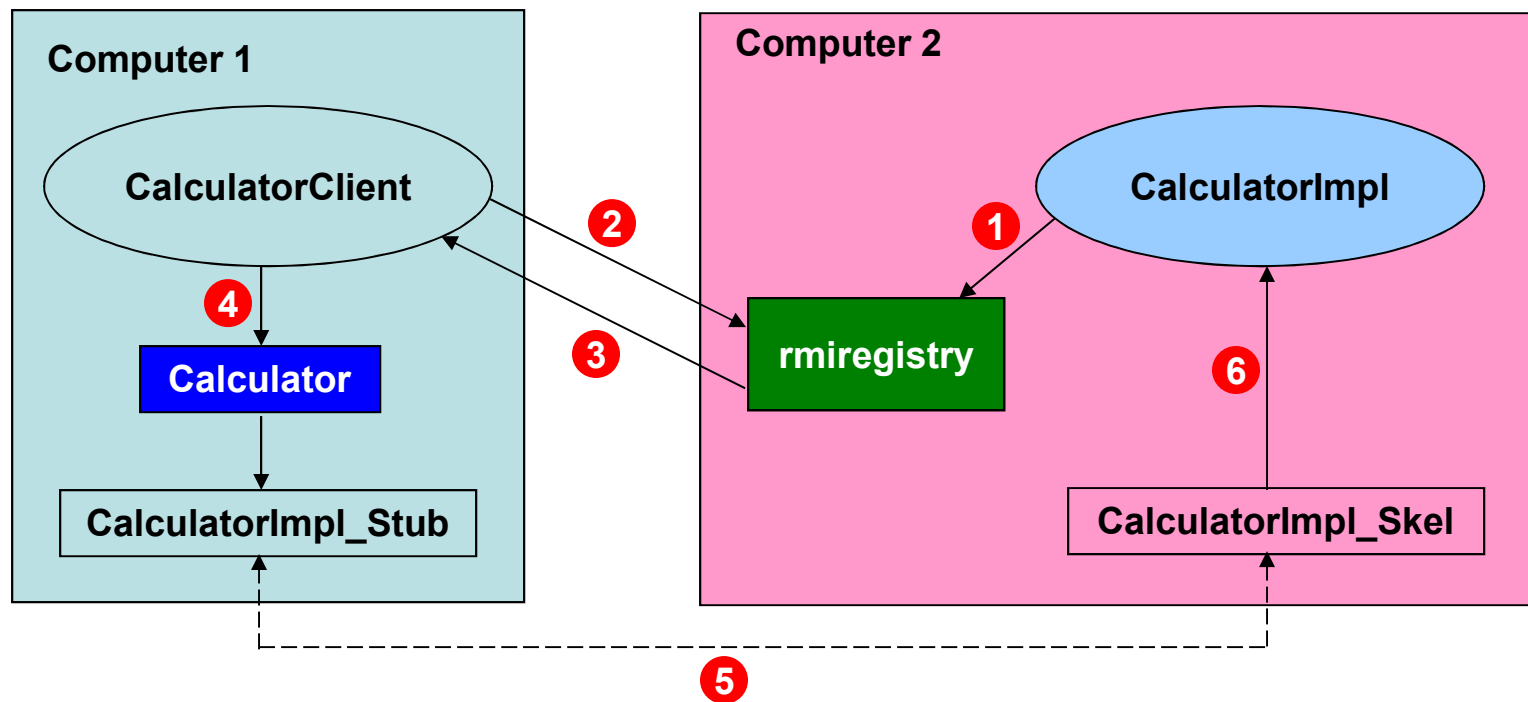
```
1 import java.rmi.*;
2
3 public class CalculatorClient{
4
5     public static void main(String args[]){
6
7         try{
8
9             System.out.println("Finding object ...");
10            Calculator c=(Calculator)Naming.lookup(
11                "rmi://localhost/MyCalculator");
12
13            System.out.println(c.addNum(12,1));
14
15        }catch (Exception e){
16            System.out.println(e);
17        }
18    }
19 }
```



**Tìm MyCalculator
trên địa chỉ localhost**

CÀI ĐẶT ỨNG DỤNG PHÂN TÁN RMI

- ✓ Diễn đạt cơ chế gọi hàm từ xa của các đối tượng RMI một cách tổng quát.

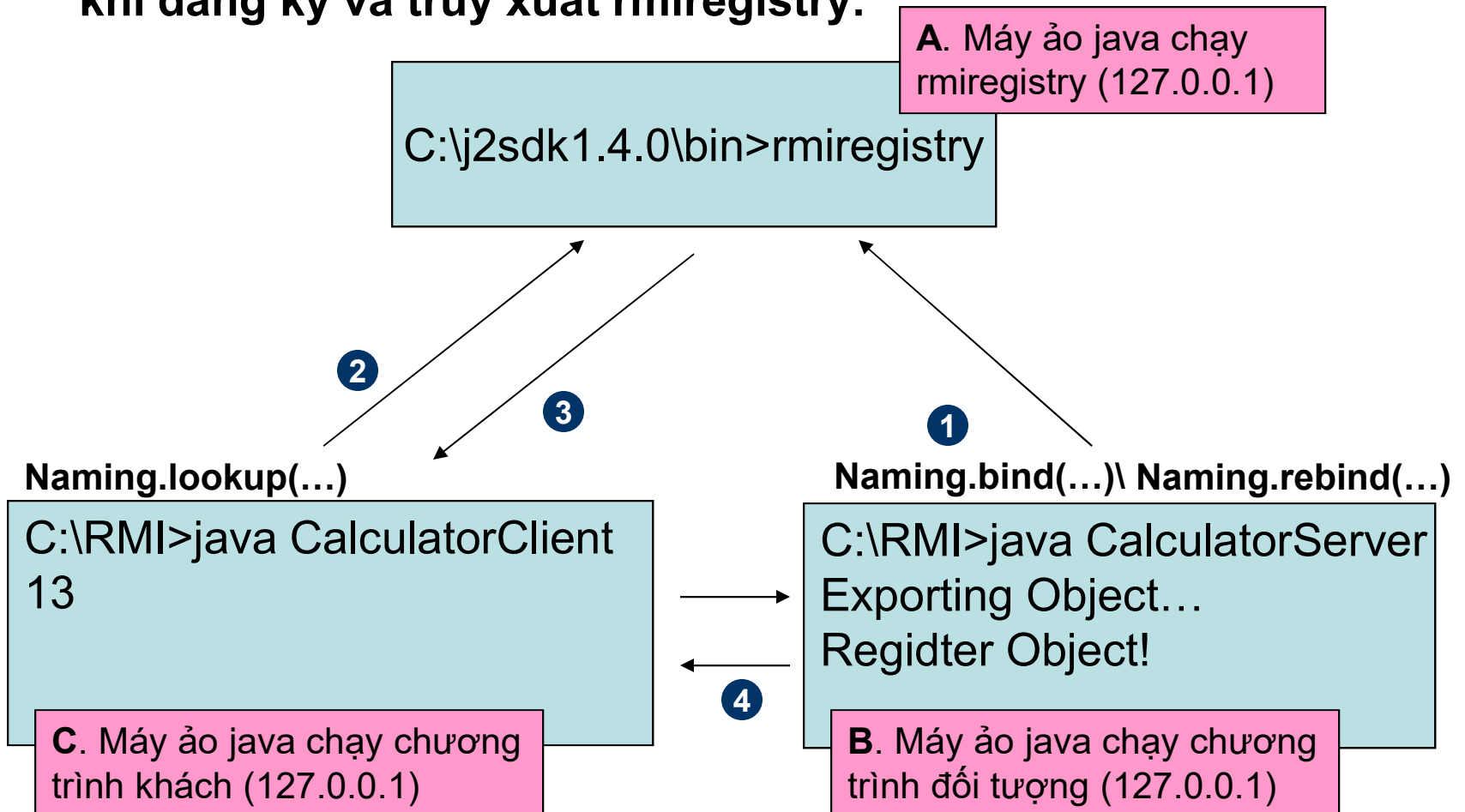


1. Đối tượng cài đặt các phương thức được tham chiếu registry sẽ trả về tham và gọi hàm Naming.bind() để đăng ký chuyển đến lớp chiếu đến đối tượng ở xa thông qua với bộ máy registry trên máy chủ. Xử lý tham số của lớp giao tiếp interface mà đối tượng
2. Đối tượng trên máy khách muốn gọi phương thức gọi đến lớp _Skel trên máy chủ.
3. Bộ tham chiếu registry sẽ trả về tham chiếu đến đối tượng ở xa thông qua với bộ máy registry trên máy chủ.
4. Dựa vào lớp giao tiếp interface đối tượng ở máy khách sẽ gọi các phương thức trên máy chủ.
5. Đối tượng cài đặt các phương thức được tham chiếu registry sẽ trả về tham và gọi hàm Naming.bind() để đăng ký chuyển đến lớp chiếu đến đối tượng ở xa thông qua với bộ máy registry trên máy chủ.
6. Đối tượng trên máy chủ sẽ trả về tham chiếu đến đối tượng ở xa thông qua với bộ máy registry trên máy chủ.

BỘ ĐĂNG KÝ (REGISTRY), CLASSPATH VÀ CODE BASE

1. Bộ đăng ký rmiregistry:

Cơ chế làm việc của trình khách và trình chủ trên máy ảo java khi đăng ký và truy xuất rmiregistry:



BỘ ĐĂNG KÝ (REGISTRY), CLASSPATH VÀ CODE BASE

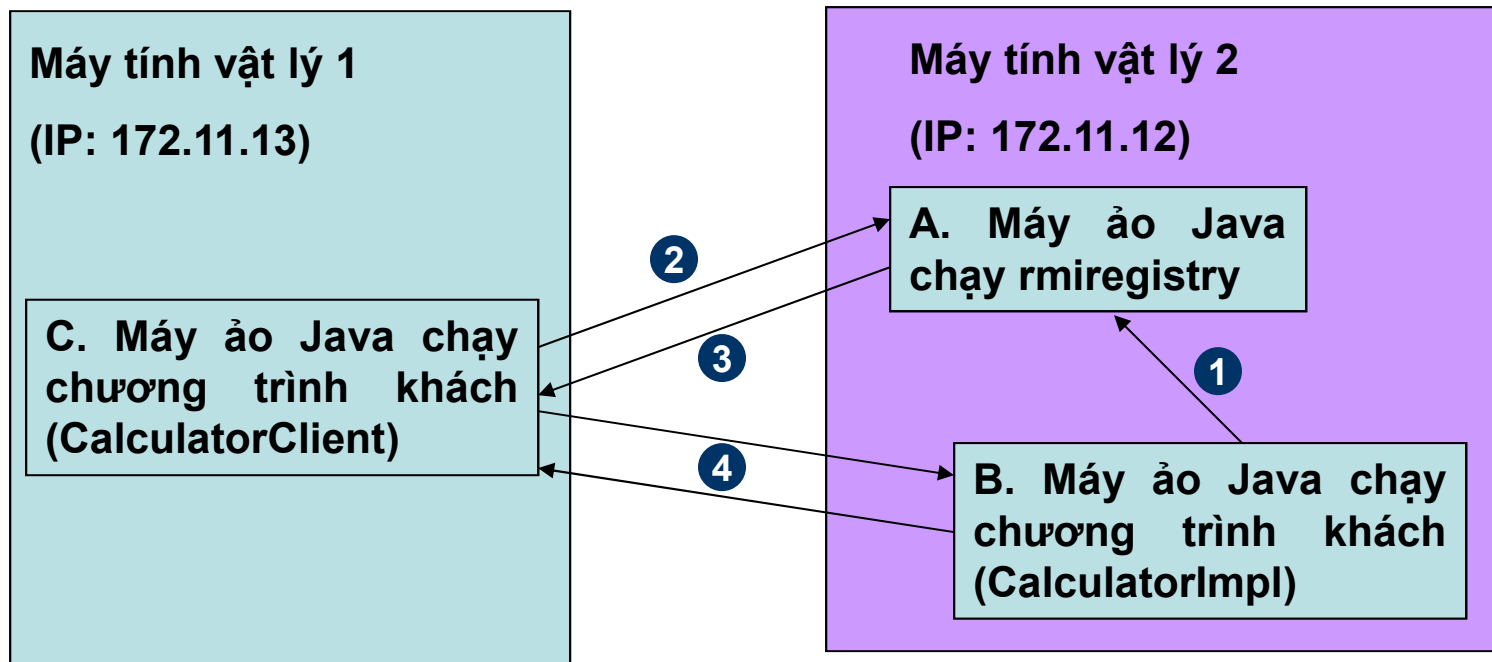
- ❶ Đối tượng trên máy ảo Java B dùng hàm **Naming.bind()** hoặc hàm **Naming.rebind()** để đăng ký đối với tượng với chương trình rmiregistry đang chạy trên máy ảo Java A.
- ❷ Chương trình khách chạy trên máy ảo Java C dùng hàm **Naming.lookup()** để yêu cầu bộ máy quản lý rmiregistry trên máy ảo A trả về tham chiếu đến đối tượng.
- ❸ Bộ quản lý trên máy ảo A trả về tham chiếu đến đối tượng đang tồn tại trên máy ảo B.
- ❹ Chương trình khách chạy trên máy ảo C sử dụng tham chiếu do hàm **Naming.lookup()** trả về để truy và gọi phương thức của đối tượng đang chạy trên máy ảo b từ xa.

Ba máy ảo này có thể xem là ba máy tính khác nhau nhưng do chạy trên cùng một máy nên IP Address giống nhau là 127.0.0.1

Thực tế mô hình trên muốn hoạt động trên mạng thật thì chỉ có thể tách ra làm hai chứ không thể tách ra ba, vì Java không cho phép rmiregistry chạy trên máy khác nơi đối tượng rmi đang hoạt động.

BỘ ĐĂNG KÝ (REGISTRY), CLASSPATH VÀ CODE BASE

Ví dụ:



Lúc này hàm đăng ký với rmiregistry được gọi như sau:

```
Naming.bind("rmi://172.11.12/Mycalculator", c) ;
```

Và hàm try tìm đối tượng từ máy khách sẽ được gọi như sau:

```
Calculator = new(Calculator)
```

```
Naming.lookup("rmi://172.11.12/Mycalculator", c) ;
```

BỘ ĐĂNG KÝ (REGISTRY), CLASSPATH VÀ CODE BASE

2. Biến Classpath và tùy chọn Codebase:

2.1 Classpath:

Khi chạy chương trình, Java dựa vào biến môi trường **CLASSPATH** để truy tìm các tập tin .class. Nếu **CLASSPATH** chỉ sai đường dẫn tới lỗi xảy ra: `java.lang.ClassNotFoundException: ClassName`

Ví dụ: Trong thư mục C:\RMI chứa hai thư mục con:

- ❑ C:\RMI\SV gồm các tập tin sau:

Calculator.class;
CalculatorImpl.class;
CalculatorServer.class;
CalculatorImpl_Stub.class;
CalculatorImpl_Skel.class

- ❑ C:\RMI\CL gồm các tập tin sau:

Calculator.class;
CalculatorClient.class;
CalculatorImpl_Stub.class;

BỘ ĐĂNG KÝ (REGISTRY), CLASSPATH VÀ CODE BASE

Khi Calculatorserver yêu cầu rmiregistry đăng ký tên đối tượng, rmiregistry sẽ đi truy tìm lớp trung gian CalculatorImpl_Stub. Trên máy ảo rmiregistry đang chạy nếu CLASSPATH không trở đến đường dẫn C:\RMI\SV thì nhận được thông báo lỗi trả về trên máy ảo nơi Calculatorserver yêu cầu đăng ký đối tượng: `java.lang.ClassNotFoundException: Calculatorimpl_Stub`

Để thay đổi giá trị cho CLASSPATH ta dùng lệnh: Set Classpath

Ví dụ: `Set Classpath=. ;C:\RMI\SV`

Giá trị mặc định của `Classpath` là trở đến thư mục chứa hiện hành. Lúc đó `Classpath=.` ;

Khi thay đổi giá trị cho `Classpath` ở một máy ảo sẽ không làm ảnh hưởng tới `Classpath` của máy ảo khác.

BỘ ĐĂNG KÝ (REGISTRY), CLASSPATH VÀ CODE BASE

2.2 Codebase:

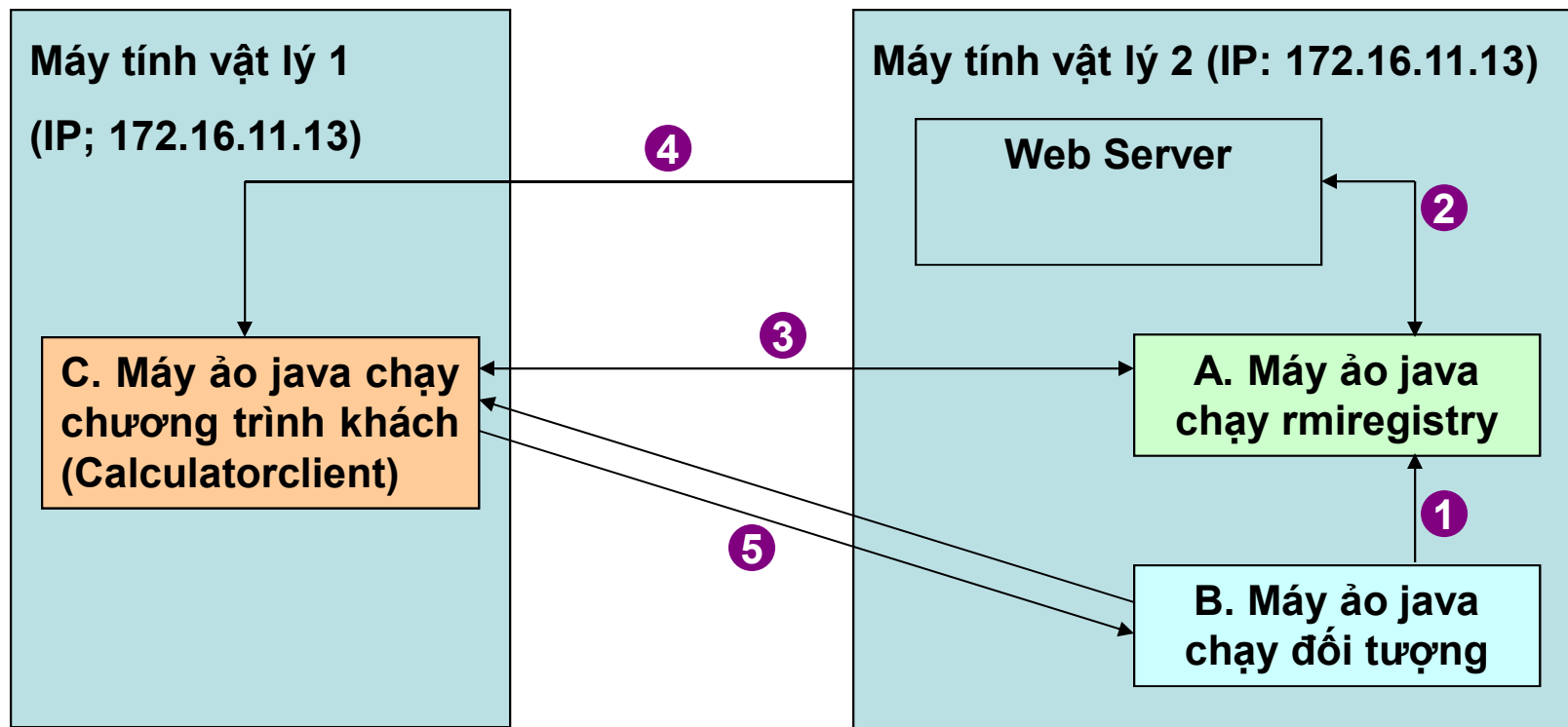
Thật sự khi viết chương trình từ phía máy khách, lập trình viên lập trình viên chỉ cần đến lớp giao tiếp `Calculator.class`. Lớp trung gian `CalculatorImpl_Stub.class` không có ý nghĩa đối với các nhà phát triển ứng dụng. Nó chỉ cần thiết cho cơ chế RMI của Java. Chính vì điều này Java cung cấp cho bạn cách thức nạp tự động lớp `CalculatorImpl_Stub.class` từ xa thông qua tùy chọn **Codebase** khi đăng ký đối tượng với `rmiregistry` trên máy chủ. Ở máy chủ phải hỗ trợ thêm dịch vụ `WebServer` chạy trên máy nơi `rmiregistry` đang chạy. Sau đó chép tập tin `CalculatorImpl_Stub.class` vào thư mục `myclass` của trình chủ web server. Khi sử dụng tùy chọn **Codebase** thì biến **Classpath** trên máy ảo nơi `rmiregistry` đang chạy không được trở đến cùng thư mục chứa `CalculatorImpl_Stub.class`. Nếu không `rmiregistry` sẽ luôn ưu tiên lấy lớp `CalculatorImpl_Stub.class` từ đường dẫn **Classpath** mà bỏ qua tùy chọn **Codebase**.

```
C:\RMI> java -Djava.rmi.server.codebase=  
"http:172.16.11.12/myclass/"calculatorServer
```

BỘ ĐĂNG KÝ (REGISTRY), CLASSPATH VÀ CODE BASE

Khi máy khách có yêu cầu rmiregistry trả về tham chiếu của đối tượng, nếu máy khách chưa có lớp Calculatorimpl_Stub.class, rmiregistry sẽ hướng dẫn máy khách tự động nạp lớp này từ địa chỉ codebase: <http://172.16.11.12/myclass/>.

Cơ chế nạp tự động lớp trung gian CalculatorImpl_Stub.class xuống máy khách.



BỘ ĐĂNG KÝ (REGISTRY), CLASSPATH VÀ CODE BASE

- 1 **Trình chủ CalculatorServer đăng ký đối tượng với rmiregistry. CalculatorServer yêu cầu bộ quản lý rmiregistry truy tìm lớp CalculatorImpl_Stub.class ở địa chỉ URL http://172.16.11.12/myclass/.**

```
C:\RMI>java -Djava.rmi.server.codebase="http://172.16.11.12/myclass"Valculatorserver
```

- 2 **Rmiregistry nhận được yêu cầu đăng ký đối tượng nó sẽ truy tìm lớp CalculatorImpl_Stub.class trong lớp Classpath. Nếu không thấy, rmiregistry sẽ dựa vào chuỗi URL do trình CalculatorServer cung cấp yêu cầu Web server trả về lớp trung gian này. Rmiregistry ghi nhớ lại địa chỉ URL nơi lớp trung gian CalculatorImpl_Stub đang được lưu trữ .**
- 3 **Chương trình ở máy khách yêu cầu emiregistry trả về tham chiếu của đối tượng. Nếu phải máy khách chưa tồn tại lớp CalculatorImpl_Stub.class bộ quản lý rmiregistry sẽ cho trình khách biết địa chỉ Url để chép lớp trung gian này về.**

BỘ ĐĂNG KÝ (REGISTRY), CLASSPATH VÀ CODE BASE

- ④** Chương trình ở máy khách yêu cầu Web Server cung cấp lớp trung gian

`http://172.16.11.12/myclass/CalculatorImpl_Stub.class`

- ⑤** Trình khách sử dụng lớp trung gian `Calculatorimpl.class` và lớp giao tiếp `Calculator.class` để gọi các phương thức của đối tượng trên máy chủ.

2.3 Nạp tập tin từ xa và chính sách bảo mật từ phía máy khách:

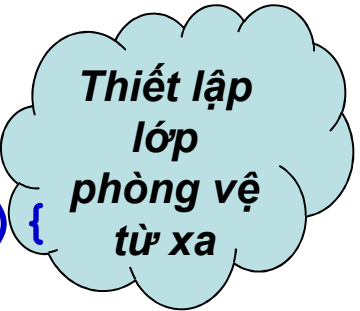
Với Java tất cả các thao tác kết nối và chép tập tin lạ từ một máy khác về một máy đều phải thông qua lớp bảo vệ gọi là `securityManager`. Để tạo lớp bảo vệ ta dùng lệnh sau:

```
System.setSecurityManager(new RMiSecurityManager());
```

Ví dụ: Mã nguồn của chương trình `CalculatorClient` trong trường hợp tự động nạp lớp `CalculatorImpl_Stub.class` từ máy chủ được viết như sau:

BỘ ĐĂNG KÝ (REGISTRY), CLASSPATH VÀ CODE BASE

```
import java.rmi.*  
  
public class CalculatorClient{  
    public static void main(String args[]){  
        try{  
            System.setSecuritymanager(new  
                RMISecurityManager());  
  
            System.out.println("Finding object...");  
            Calculator c = CalculatorNaming.lookup(  
                "rmi://172.16.11.12/myCalculator");  
  
            .....  
        }catch (Exception e) {  
            System.out.println(e);  
        }  
    }  
}
```



Thiết lập
lớp
phòng vệ
từ xa

CHUYỂN THAM SỐ TRONG LỜI GỌI PHƯƠNG THỨC TỪ XA

❖ Chuyển tham số theo tham trị và tham biến:

Đối với Java, trên máy cục bộ hầu hết các biến kiểu đối tượng đều truyền theo tham chiếu trong các lời gọi hàm. Nghĩa là một khi biến đối tượng được truyền vào phương thức nếu bên trong phương thức thay đổi giá trị của đối tượng thì khi lời gọi phương thức chấm dứt giá trị của đối tượng cũng thay đổi theo. Ví dụ:

CHUYỂN THAM SỐ TRONG LỜI GỌI PHƯƠNG THỨC TỪ XA

```
1  class Number{
2      public int value = 0;
3      public number(int v){
4          value = v;
5      }
6
7  }
8
9
10 public class Program{
11     public static void main(String args[]){
12         Number num = new Number(12);
13         doIncrease(num);
14         System.out.println(num.value);
15     }
16     public static void doIncrease(Number n){
17         n.value++;
18     }
19 }
```


CHUYỂN THAM SỐ TRONG LỜI GỌI PHƯƠNG THỨC TỪ XA

Khi chạy chương trình value sẽ có giá trị là 13. Tuy nhiên các kiểu dữ liệu đơn giản như: int, float, double, char, byte, long... lại được truyền theo tham trị. Giá trị của tham số mà hàm hay phương thức xử lý chỉ là bản sao của biến truyền từ ngoài vào. Ví dụ:

```
1 public class Program{
2     public static void main(String args[]){
3         int num = 12;
4         doIncrease(num);
5         System.out.println(num);
6     }
7     public static void doIncrease(int){
8         n++;
9     }
10 }
```

Ở ví dụ này kết quả xuất ra màn hình là 12. Tóm lại, trong java đối tượng được truyền theo tham chiếu còn các kiểu dữ liệu đơn giản được truyền theo tham trị.

CHUYỂN THAM SỐ TRONG LỜI GỌI PHƯƠNG THỨC TỪ XA

Việc truyền tham số qua mạng theo cơ chế RMI lại khác với cách truyền tham số thông thường.

Tất cả các kiểu dữ liệu đơn giản như int,char... đều được truyền theo tham trị.

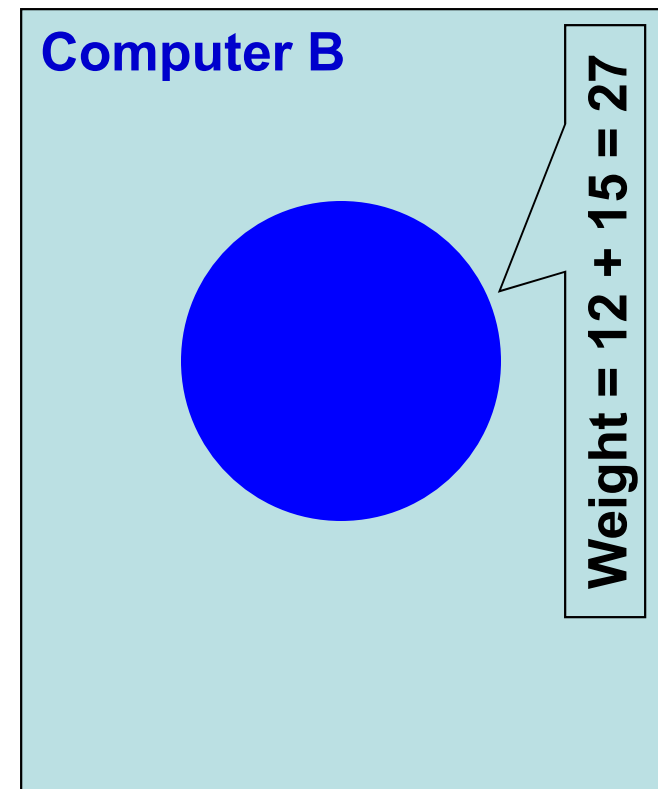
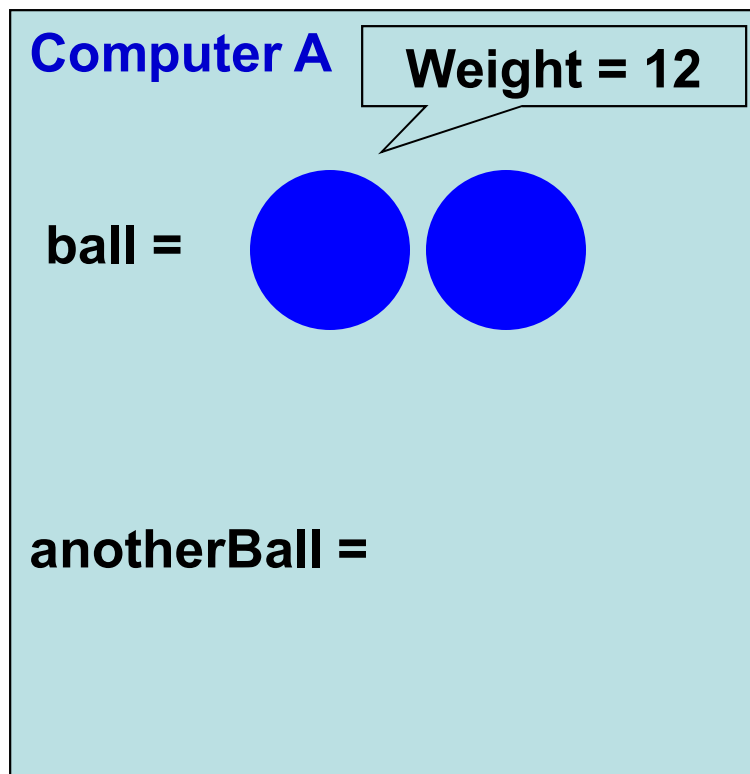
Tất cả các dữ liệu kiểu đối tượng muốn truyền qua mạng đều buộc phải cài đặt một trong hai giao tiếp **Remote** hoặc **Serializable**.

Các đối tượng cài đặt giao tiếp **Remote** sẽ được truyền theo tham chiếu còn các đối tượng cài đặt giao tiếp **Serializable** sẽ được truyền theo tham trị.

CHUYỂN THAM SỐ TRONG LỜI GỌI PHƯƠNG THỨC TỪ XA

Ví dụ về truyền tham số qua mạng sử dụng giao tiếp Serializable:

Mô phỏng ví dụ:



CHUYỂN THAM SỐ TRONG LỜI GỌI PHƯƠNG THỨC TỪ XA

Ví dụ về truyền tham số qua mạng sử dụng giao tiếp Serializable:
// Ball.java.

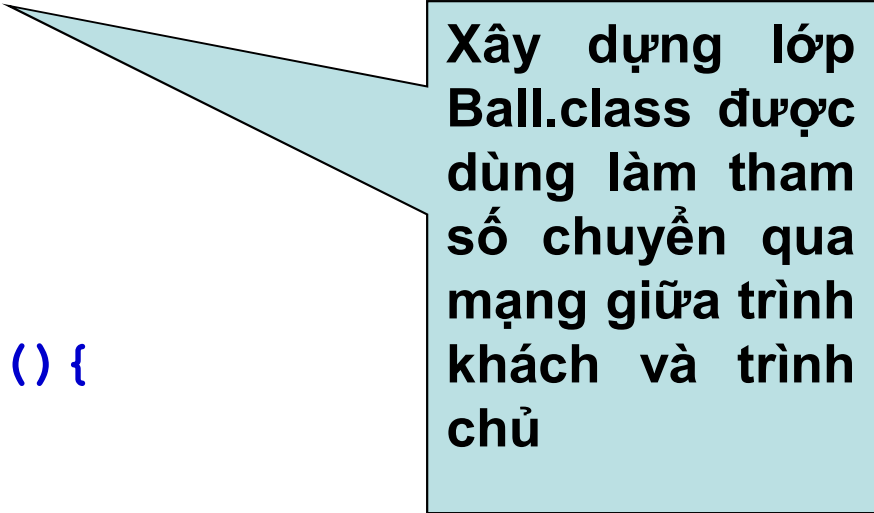
```
import java.io.*;

public class Ball implements Serializable{
    int weight=0;

    public Ball(int w){
        weight=w;
    }

    public int getWeight(){
        return weight;
    }

    public void setWeight(int w){
        weight=w;
    }
}
```



Xây dựng lớp Ball.class được dùng làm tham số chuyển qua mạng giữa trình khách và trình chủ

CHUYỂN THAM SỐ TRONG LỜI GỌI PHƯƠNG THỨC TỪ XA

//PingServer.java

```
import java.rmi.*;

public interface PingServer
extends Remote {

    public Ball ping(Ball b)
    throws RemoteException;

}
```



Đặc tả giao tiếp
interface cho
đối tượng trên
máy chủ

CHUYỂN THAM SỐ TRONG LỜI GỌI PHƯƠNG THỨC TỪ XA

//PingServerImpl.java

```
import java.rmi.*;
```

```
public class PingServerImpl implements PingServer{
```

```
    public Ball ping(Ball b) throws RemoteException{  
        System.out.println("Client send a ball  
        objectweight" + b.getWeiht());
```

```
        b.setWeight(b.getWeight()+15);
```

```
        return b;
```

```
}
```

Cài đặt chi tiết cho đối tượng thông qua đối lớp PingServerImpl

CHUYỂN THAM SỐ TRONG LỜI GỌI PHƯƠNG THỨC TỪ XA

//Setup.java

```
import java.rmi.*;  
import java.rmi.server.*;
```

```
public class Setup {  
    public static void main(String args[]) throws  
                                   Exception{  
        PingServer server=new PingServerImpl();  
        UnicastRemoteObject.exportObject(server);  
  
        Naming.bind("rmi://localhost/pingobject",server);  
  
        System.out.println("Waiting for client request  
...");  
    }  
}
```

Chương trình Setup cài đặt đối tượng PingServerImpl trên máy chủ

CHUYỂN THAM SỐ TRONG LỜI GỌI PHƯƠNG THỨC TỪ XA

```
//Client.java
import java.rmi.*;

public class Client {
    public static void main(String args[]) throws Exception{

        Ball ball=new Ball(12);

        PingServer server=(PingServer)Naming.lookup(
            "rmi://localhost/pingobject");

        System.out.println("Ball weight before send to server
            "+ball.getWeight());

        Ball anotherBall=server.ping(ball);

        System.out.println("Ball weight after send to server
            "+ball.getWeight());

        System.out.println("Ball weight return by server
            "+anotherBall.getWeight());

    }
}
```

Chương trình Client gọi phương thức của đối tượng pingServer

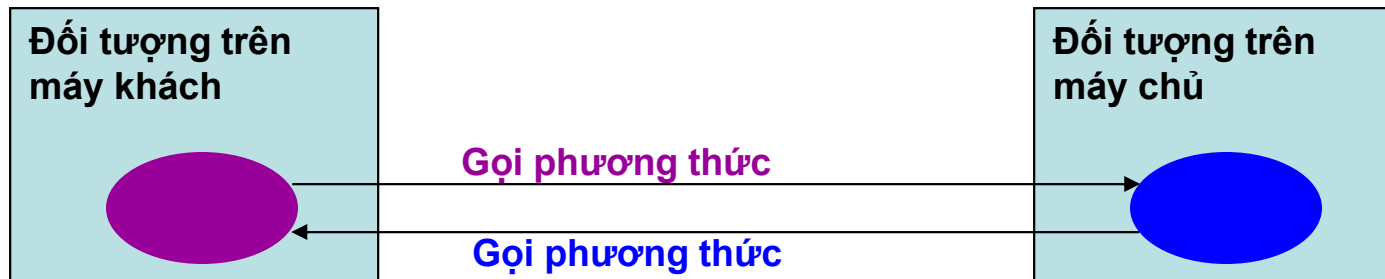
CHUYỂN THAM SỐ TRONG LỜI GỌI PHƯƠNG THỨC TỪ XA

Trong trình khách Client.java đối tượng ball trước và sau gọi phương thức ping() vẫn giữ nguyên giá trị là 12. Trong khi đó đối tượng ball đưa lên trình chủ được tăng giá trị lên 15. Kết quả trả về từ trình chủ là một đối tượng anotherBall khác với đối tượng ball được chuyển đi từ trình khách – anotherBall có giá trị là $12 + 15 = 27$. Như vậy đối tượng ball được phương thức ping() chuyển đi theo tham trị.

Nếu đối tượng ball quá lớn việc đóng gói toàn bộ đối tượng chuyển đi chuyển lại trên mạng sẽ ảnh hưởng đến tốc độ thực thi của chương trình.

Bây giờ chúng ta sẽ xây dựng chương trình có thể tham chiếu và xử lý trực tiếp đối tượng nằm trên máy khách. Nghĩa là nếu trình chủ được trình khách truy xuất từ xa thì trình khách cũng được gọi từ xa bởi trình chủ. Bằng cách này trình chủ và trình khách có thể triệu gọi lẫn nhau. (*không như trước giờ ta vẫn theo cơ chế thụ động trình khách gọi phương thức của trình chủ*).

CHUYỂN THAM SỐ TRONG LỜI GỌI PHƯƠNG THỨC TỪ XA



Cơ chế gọi ngược từ xa của trình chủ đến trình khách thông qua tham chiếu gọi là cơ chế callback.

Ví dụ:

```
//AtClient.java
import java.rmi.*;

public interface AtClient extends Remote {

    public void callClientMethod(String message)
        throws RemoteException;
}
```

Đặt tả giao tiếp interface cho đối tượng AtClient.

CHUYỂN THAM SỐ TRONG LỜI GỌI PHƯƠNG THỨC TỪ XA

//AtServer.java

```
import java.rmi.*;
```

Đặt tả giao tiếp interface cho đối tượng AtServer.

```
public interface AtServer extends Remote {  
    public void registerClient(AtClient c)  
        throws RemoteException;  
  
    public void callServerMethod(String message)  
        throws RemoteException;  
}
```

Phương thức registerClient() dùng để tiếp nhận tham chiếu đến đối tượng AtClient. Phương thức callServerMethod() dùng để cung cấp các dịch vụ cho trình khách.

CHUYỂN THAM SỐ TRONG LỜI GỌI PHƯƠNG THỨC TỪ XA

Cài đặt chi tiết cho đối tượng
AtClient thông qua lớp
AtClientImpl

```
//AtClientImpl.java  
import java.rmi.*;
```

```
public class AtClientImpl implements AtClient {  
  
    public void callClientMethod(String message)  
                                throws RemoteException{  
        System.out.println(message) ;  
    }  
}
```

CHUYỂN THAM SỐ TRONG LỜI GỌI PHƯƠNG THỨC TỪ XA

//AtServerImpl.java

```
import java.rmi.*;
public class AtServerImpl implements AtServer {

    AtClient client; // lưu giữ tham chiếu đến đối tượng trên máy khách

    public void registerClient(AtClient c) throws
                                   RemoteException{
        client = c;
    } // Được trình khách triệu gọi để đăng ký đối tượng AtClient với trình chủ

    public void callServerMethod(String message) throws
                                   RemoteException{

        System.out.println(message) ;

        for (int i=1; i<10; i++){
            String msg= "Server response "+ Math.random()*1000;
            // Triệu gọi phương thức của đối tượng chạy trên máy khách
            client.callClientMethod(msg) ;
        }
    }
}
```

}

CHUYỂN THAM SỐ TRONG LỜI GỌI PHƯƠNG THỨC TỪ XA

//Client.java

```
import java.rmi.*;
import java.rmi.server.*;

public class Client {

    public static void main(String args[]) throws
                                                Exception{

        AtClient client=new AtClientImpl();

        UnicastRemoteObject.exportObject(client);

        AtServer server=(AtServer)Naming.lookup(
            "rmi://localhost/serverobject");

        server.registerClient(client);
        server.callServerMethod("Client Contact
                                Server");
    }
}
```

CHUYỂN THAM SỐ TRONG LỜI GỌI PHƯƠNG THỨC TỪ XA

Do java cho phép một đối tượng có thể cài đặt cùng lúc nhiều giao tiếp interface khác nhau nên đối tượng có thể cài đặt cùng lúc hai giao tiếp Serializable và Remote.

Khi đó nếu không dùng `UnicastRemoteObject.exportObject()` để đăng ký đối tượng cho máy ảo Java thì đối tượng sẽ được truyền theo tham trị, ngược lại đối tượng sẽ được truyền theo tham chiếu.

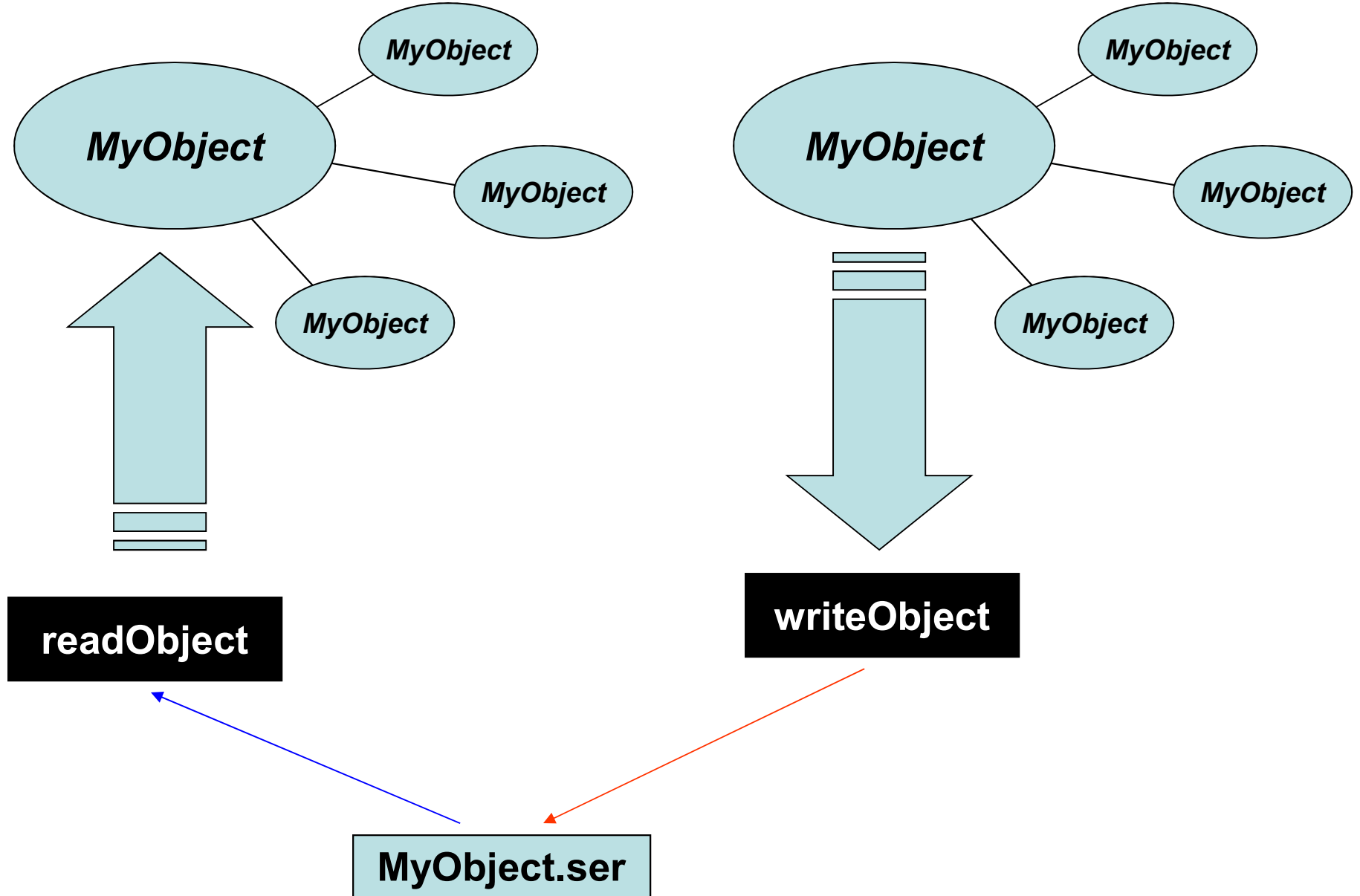
TUẦN TỰ HÓA (SERIALIZABLE) ĐỐI TƯỢNG

Java cung cấp cho bạn khả năng ghi toàn bộ đối tượng xuống một tập tin. Sau đó bạn có thể đem tập tin chứa đối tượng đi khắp nơi nếu cần thì khôi phục đối tượng về trạng thái ban đầu. Đối tượng trong bộ nhớ có thể quan hệ chằng chịt với nhau nhưng khi được ghi xuống tập tin thì bắt buộc phải trải phẳng đối tượng theo thứ tự.

Trong Java đối tượng muốn có khả năng ghi và lưu dữ liệu thành tập tin phải cài đặt giao tiếp `java.io.Serializable`. Nếu đối tượng cần ghi chứa bên trong nhiều đối tượng con khác thì Java lấy tất cả các đối tượng con ghi luôn xuống đĩa.

Dưới đây là ví dụ cho thấy khả năng tuần tự hóa của một đối tượng phức hợp.

TUẦN TỰ HÓA (SERIALIZABLE) ĐỐI TƯỢNG



TUẦN TỰ HÓA (SERIALIZABLE) ĐỐI TƯỢNG

```
2  import java.io.*;
3
4  class LinkObject implements Serializable{
5
6      static int count=1;
7      LinkObject next;
8      int id;
9
10     public LinkObject(int id){
11         this.id=id;
12         if (count<50) {
13             count++;
14             next=new LinkObject(count);
15         } else next=null;
16     }
17
18     public String toString(){
19         return ("Object "+id);
20     }
21 }
```

Lớp đối tượng lưu thành tập trên đĩa.

TUẦN TỰ HÓA (SERIALIZABLE) ĐỐI TƯỢNG

Tạo file để lưu đối tượng

Khởi tạo đối tượng trong bộ nhớ

// Ghi đối tượng xuống tập tin

```
23 public class WriteObjectApp{
24
25     public static void main(String args[]){
26         try{
27             ObjectOutputStream out =new ObjectOutputStream(
28                 new FileOutputStream("myobject.ser"));
29
30
31             LinkObject X=new LinkObject(1);
32             System.out.println("X is "+X);
33
34             out.writeObject(X); // Ghi đối tượng xuống tập tin
35
36             System.out.println("Here is object content");
37             while ((X=X.next)!=null)
38                 System.out.println(X);
39         } catch(FileNotFoundException e){e.getMessage();}
40
41         catch(IOException e){e.getMessage();}
42
43     }
44 }
```

TUẦN TỰ HÓA (SERIALIZABLE) ĐỐI TƯỢNG

*Mở tập tin
chứa đối
tượng*

*Đọc
đối
tượng
từ tập
tin*

```
2  import java.io.*;
3
4  public class ReadObjectApp{
5      public static void main(String args[]){
6          try{
7
8              ObjectInputStream in = new ObjectInputStream(
9                  new FileInputStream("myobject.ser"));
10
11              LinkObject X = (LinkObject) in.readObject();
12
13              System.out.println("Here is X "+X);
14
15              while ((X=X.next()) != null)
16                  System.out.println(X);
17          }catch(FileNotFoundException e){e.getMessage();}
18
19          catch(IOException e){e.getMessage();}
20
21          catch(ClassNotFoundException e){e.getMessage();}
22
23      }
24  }
```

TÌM HIỂU RMI REGISTRY VÀ CÁC CÁCH ĐĂNG KÝ ĐỐI TƯỢNG

Trước giờ ta vẫn sử dụng bộ đăng ký rmiregistry như một dịch vụ tìm kiếm. Các đối tượng chủ muốn được trình khách truy tìm được từ xa thì trước hết phải đăng ký với rmiregistry. Bộ đăng ký này là một chương trình dịch vụ chạy ở hậu trường (chương trình rmiregistry.exe) mở ổ cắm socket và lắng nghe các yêu cầu gửi đến cổng mặc định 1099, bạn có thể chỉ định một cổng khác với cổng 1099.

Ví dụ: `C:\j2sdk1.4.0\bin\rmiregistry.exe 9999`

lúc này rmiregistry sẽ chạy trên cổng 9999, và đối tượng trên máy chủ sẽ đăng ký với rmiregistry theo cách sau:

```
Myobject o = new Myobject();
```

```
Naming.bind("rmi://localhost:9999/myObjectName",o);
```

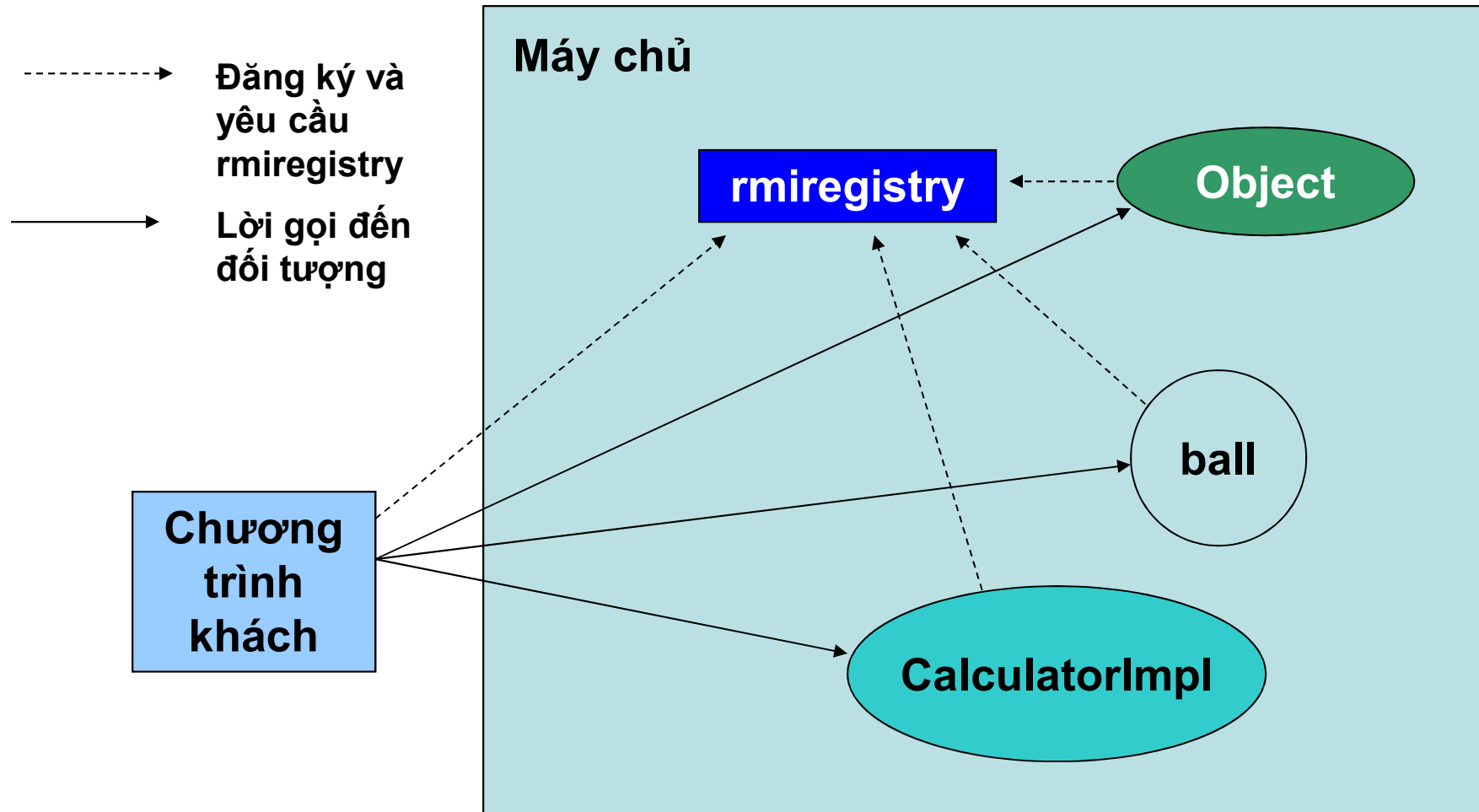
tiếp đến chương trình trên máy khách sẽ yêu cầu dịch vụ rmiregistry trả về tham chiếu đến đối tượng trên máy chủ:

```
Myobject o =
```

```
(Myobject)Naming.lookup("rmi://localhost:9999/myObjectna  
me",o);
```

TÌM HIỂU RMI REGISTRY VÀ CÁC CÁCH ĐĂNG KÝ ĐỐI TƯỢNG

Một dịch vụ đăng ký có thể tiếp nhận và quản lý cùng lúc nhiều đối tượng chủ khác nhau.



TÌM HIỂU RMI REGISTRY VÀ CÁC CÁCH ĐĂNG KÝ ĐỐI TƯỢNG

Trong java cũng cho phép chúng ta lấy về danh sách các đối tượng mà rmi registry đang nắm giữ . Dưới đây là chương trình tìm hiểu danh sách tên các đối tượng do rmi registry nắm giữ.

```
2  import java.rmi.registry.*;
3
4  public class Traverse{
5      public static void main(String args[]) throws Exception{
6          String hostAddr="127.0.0.1";//localhost
7
8          System.out.println("Connecting registry...");
9          Registry registry=
10              LocateRegistry.getRegistry(hostAddr);
11
12          String objectAvailable[]=registry.list();
13
14          System.out.println("Registry object: ");
15
16          for (int i=0; i<objectAvailable.length; i++){
17              System.out.println(objectAvailable[i]);
18          }
19      }
20  }
```

Kết
nối với
bộ
đăng

kí

Lấy
về
danh
sách
các
đối
tượng

TÌM HIỂU RMI REGISTRY VÀ CÁC CÁCH ĐĂNG KÝ ĐỐI TƯỢNG

Trong Java cho phép bạn tự tạo bộ đăng kí mà không cần dùng đến chương trình **rmiregistry.exe**. Để tạo bộ đăng kí và tự đăng kí đối tượng chúng ta gọi phương thức tĩnh **createRegistry()** của lớp **LocateRegistry**. Ví dụ:

```
2 import java.rmi.*;
3 import java.rmi.server.*;
4 import java.rmi.registry.*;
5
6 public class Setup{
7     public static void main(String args[])throws Exception{
8
9
10         LocateRegistry.createRegistry(1099);
11
12
13         Calculator cal=new CalculatorImpl();
14
15
16         UnicastRemoteObject.exportObject(cal);
17
18
19         System.out.println("Registering object ...");
20         Naming.bind("rmi://localhost/myCal",cal);
21         System.out.println("Waiting for client request...");
22
23     }
24 }
```

TÌM HIỂU RMI REGISTRY VÀ CÁC CÁCH ĐĂNG KÝ ĐỐI TƯỢNG

Tuy nhiên cách này chỉ có thể áp dụng cho một đối tượng. Nếu một đối tượng nào đó đã tạo ra bộ đăng kí rồi thì bạn không gọi được `LocateRegistry.createRegistry()` lần thứ hai trên cùng một cổng của máy chủ. Ví dụ:

```
LocateRegistry.createRegistry(1234) ;
```

```
.....
```

```
Naming.bind("rmi://localhost:1234/myCal",cal) ;
```

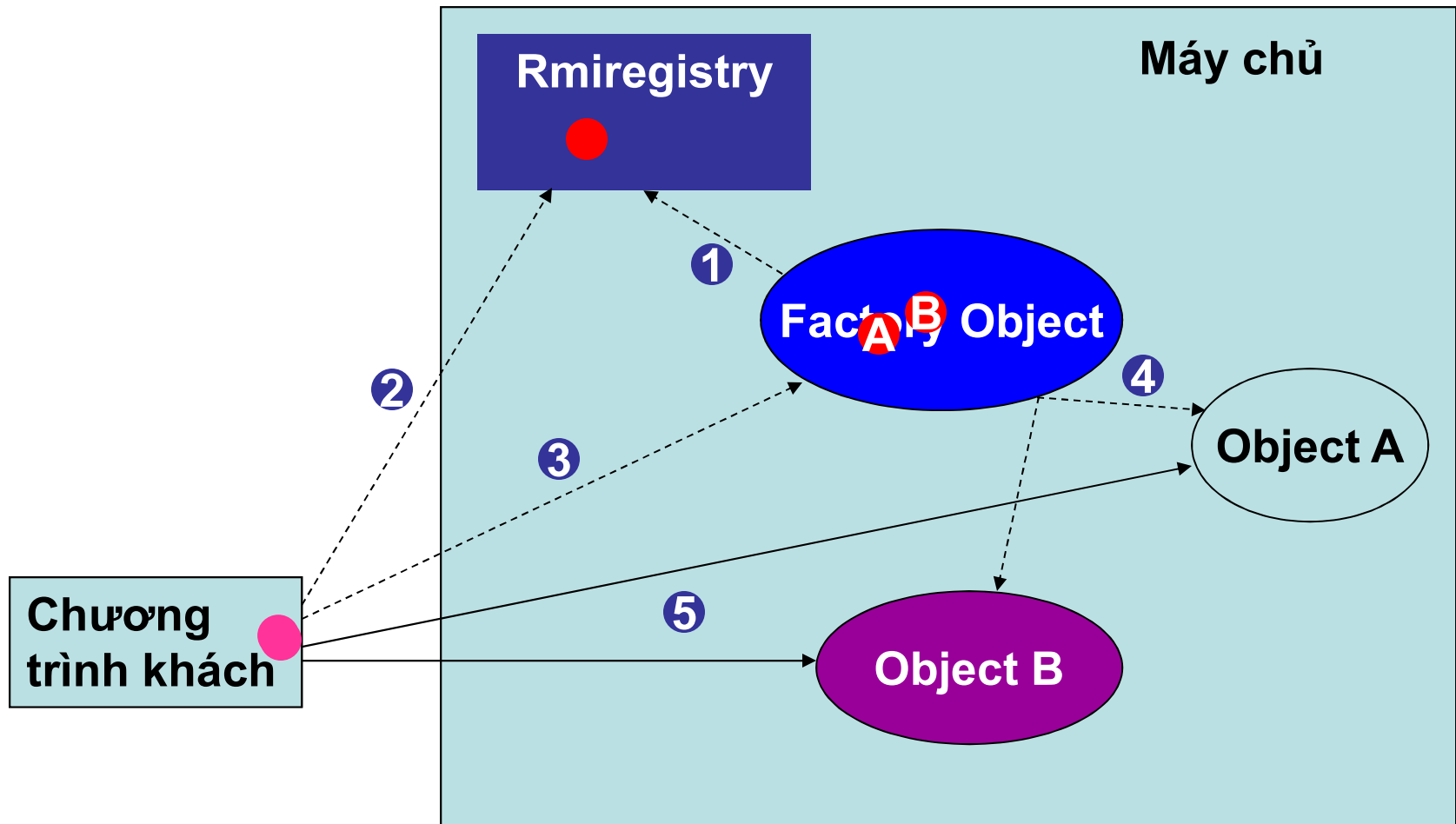
DÙNG MỘT ĐỐI TƯỢNG SẢN SINH NHIỀU ĐỐI TƯỢNG (FACTORYOBJECT)

1. Mô hình xưởng chế tác đối tượng:

Cho đến bây giờ mỗi khi xây dựng một đối tượng chủ chúng ta đều phải đặt tên cho đối tượng, đăng kí tên đối tượng cho registry. Điểm bất tiện là registry phải quản lý quá nhiều tên đối tượng và khi viết chương trình trên máy khách chúng ta cũng phải nhớ tên của các đối tượng này. Để tiện lợi hơn java cho phép chúng ta xây dựng một đối tượng duy nhất và đăng kí đối tượng này với registry, nhiệm vụ của nó chỉ dùng để tạo ra các đối tượng con khác. Một đối tượng như vậy trong mô hình hướng đối tượng gọi là factory object.

Dưới đây là mô hình hoạt động của đối tượng Factory

DÙNG MỘT ĐỐI TƯỢNG SẢN SINH NHIỀU ĐỐI TƯỢNG (FACTORYOBJECT)



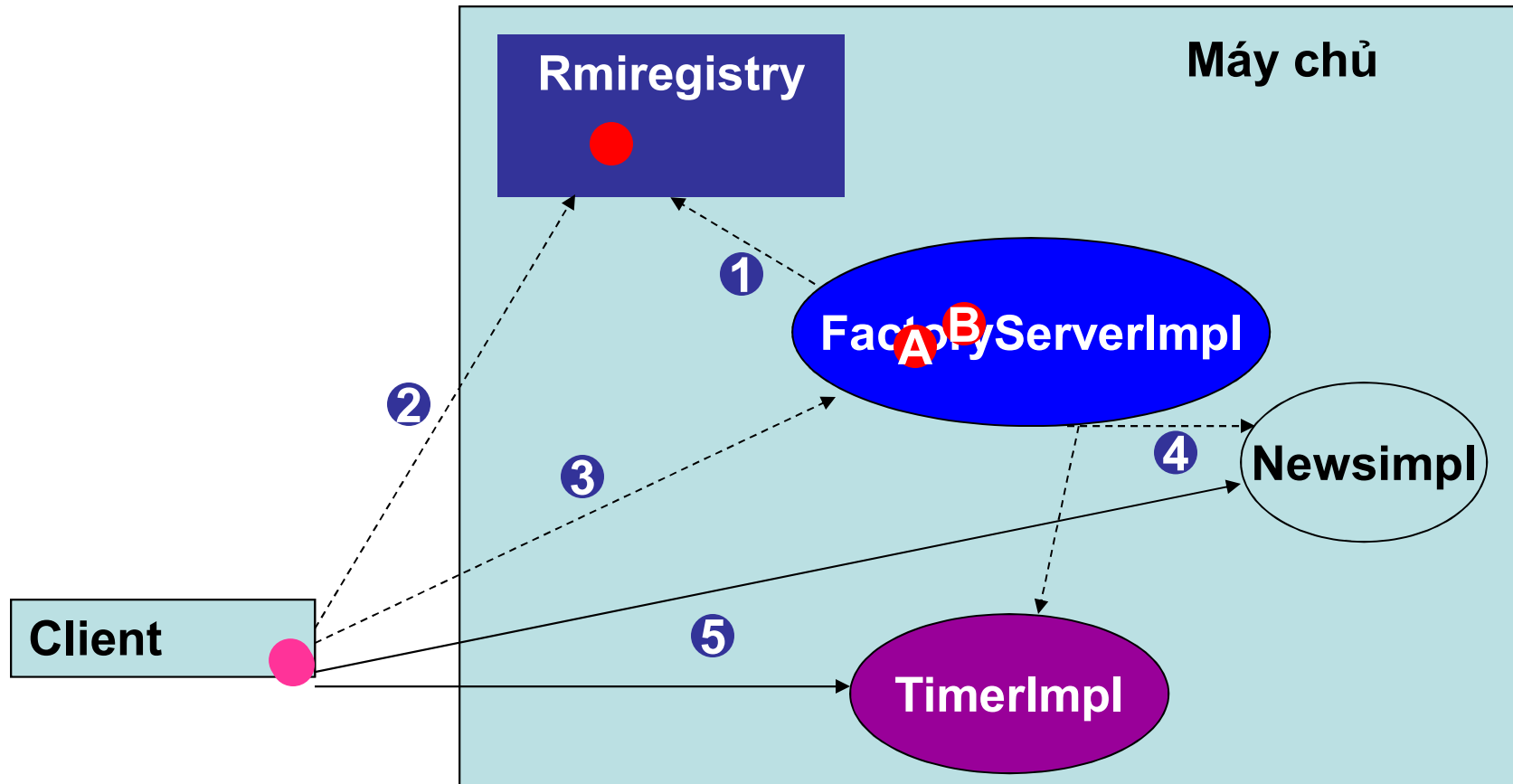
DÙNG MỘT ĐỐI TƯỢNG SẢN SINH NHIỀU ĐỐI TƯỢNG (FACTORYOBJECT)

- ① Đối tượng Factory Object đăng ký tên mình với rmiregistry.**
- ② Trình khách muốn gọi Object A. Trước hết trình khách liên hệ với rmiregistry để lấy về tham chiếu đến đối tượng Factory Object.**
- ③ Sau khi có tham chiếu của đối tượng Factory Object, trình khác triệu gọi đối tượng Factory Object yêu cầu tạo ra và cho phép tham chiếu đến Object A hoặc Object B.**
- ④ Đối tượng chủ tạo ra và trả về tham chiếu của Object A, Object B theo yêu cầu của trình khách.**
- ⑤ Dựa vào tham chiếu do Factory Object trả về trình khách thực hiện lời gọi đến Object A hoặc Object B.**

DÙNG MỘT ĐỐI TƯỢNG SẢN SINH NHIỀU ĐỐI TƯỢNG (FACTORYOBJECT)

2. Cài đặt ứng dụng Factory:

Giả sử chúng ta có đối tượng **FactoryServerImpl** dùng để tạo ra hai đối tượng con: **NewsImpl** dùng để lấy về thông tin và **TimerImpl** dùng để lấy ngày. Chúng ta có mô hình sau:



DÙNG MỘT ĐỐI TƯỢNG SẢN SINH NHIỀU ĐỐI TƯỢNG (FACTORYOBJECT)

//Timer.java

```
2 import java.rmi.*;
3 import java.util.*;
4
5 public interface Timer extends Remote{
6     public Date getCurrentTime() throws RemoteException;
7 }
```

//TimerImpl.java

```
2 import java.rmi.*;
3 import java.util.*;
4
5 public class TimerImpl implements Timer{
6
7     public Date getCurrentTime() throws RemoteException{
8
9         System.out.println("Client request time");
10
11         return new Date();
12     }
13 }
```

DÙNG MỘT ĐỐI TƯỢNG SẢN SINH NHIỀU ĐỐI TƯỢNG (FACTORYOBJECT)

```
2  import java.rmi.*; //News.java
3
4  public interface News extends Remote{
5      public String getTodayNews() throws RemoteException;
6  }
```

```
2  import java.rmi.*; //NewsImpl.java
3
4  public class NewsImpl implements News{
5      String newslst[]={
6          "Sun released new Java version",
7          "Microsoft announce Windows 2003",
8          "Fujitsu setup new factory in Vietnam"};
9
10     static int index=0;
11
12     public String getTodayNews() throws RemoteException{
13
14         System.out.println("Client request news");
15
16         index++;
17         if (index>2) index=0;
18         return newslst[index];
19     }
20 }
21
```


DÙNG MỘT ĐỐI TƯỢNG SẢN SINH NHIỀU ĐỐI TƯỢNG (FACTORYOBJECT)

//FactoryService.java

```
2  import java.rmi.*;
3
4  public interface FactoryService extends Remote{
5      public News  createNews() throws RemoteException;
6
7      public Timer createTimer() throws RemoteException;
8  }
```

DÙNG MỘT ĐỐI TƯỢNG SẢN SINH NHIỀU ĐỐI TƯỢNG (FACTORYOBJECT)

//FactoryServiceImpl.java

```
2  import java.rmi.*;
3  import java.rmi.server.*;
4
5  public class FactoryServiceImpl implements FactoryService{
6
7
8      News  newsobject =new NewsImpl();
9      Timer timerobject= new TimerImpl();
10
11  public FactoryServiceImpl(){
12      try {
13          UnicastRemoteObject.exportObject(newsobject);
14          UnicastRemoteObject.exportObject(timerobject);
15      } catch (Exception e){
16          System.out.println(e);
17      }
18  }
19
20  public News  createNews() throws RemoteException{
21
22      return newsobject;
23  }
24
25  public Timer createTimer() throws RemoteException{
26      return timerobject;
27  }
28 }
```

DÙNG MỘT ĐỐI TƯỢNG SẢN SINH NHIỀU ĐỐI TƯỢNG (FACTORYOBJECT)

//Setup.java

```
2  import java.rmi.*;
3  import java.rmi.server.*;
4
5  public class Setup{
6
7      public static void main(String args[])throws Exception{
8
9          System.out.println("Factory object created ");
10
11          FactoryService obj=new FactoryServiceImpl();
12
13          UnicastRemoteObject.exportObject(obj);
14
15          Naming.bind("rmi://localhost/myfactory",obj);
16
17      }
18 }
```

DÙNG MỘT ĐỐI TƯỢNG SẢN SINH NHIỀU ĐỐI TƯỢNG (FACTORYOBJECT)

//Client.java

```
2 import java.rmi.*;
3 import java.rmi.server.*;
4
5 public class Client{
6     public static void main(String args[])throws Exception{
7
8         FactoryService service=(FactoryService)
9             Naming.lookup("rmi://localhost/myfactory");
10
11         News news=service.createNews();
12         Timer timer=service.createTimer();
13
14         System.out.println("Current "+timer.getCurrentTime());
15         System.out.println("We have news "+ news.getTodayNews());
16
17         Thread.sleep(1000);
18
19         System.out.println("Current "+timer.getCurrentTime());
20         System.out.println("We have news "+news.getTodayNews());
21     }
22 }
23
```

KỸ THUẬT GỌI ĐỐI TƯỢNG Ở XA BẰNG PHƯƠNG PHÁP ĐỘNG (DYNAMIC METHOD INVOKE)

.....